

Software-Defined Radio on Raspberry Pi

Subhodeep Chakraborty and G.V.V. Sharma
Department of Electrical Engineering
IIT Hyderabad
Email: gadepall@ee.iith.ac.in

Abstract—This paper details the implementation of a bare-metal Software-Defined Radio (SDR) utilising a standard Raspberry Pi as an RF transmitter and an RTL-SDR as a receiver. Building upon existing hardware analysis of the Pi-FM architecture, this work introduces the mathematical and programmatic generation of both analog (AM/FM) and digital (16-QAM) Baseband I/Q signals using Python. While the software-defined theoretical execution of the 16-QAM baseband was flawless, real-world over-the-air transmission demonstrated severe signal degradation.

Index Terms—Software-Defined Radio, Raspberry Pi, 16-QAM, Multipath Fading, AM/FM Modulation, I/Q Baseband.

I. INTRODUCTION

The democratisation of Software-Defined Radios (SDR) has allowed standard computing hardware to replace specialised analog circuits. Using software like `rpitx` and Pi-FM, a standard Raspberry Pi can bypass traditional Digital-to-Analog Converters (DAC) and utilise Direct Memory Access (DMA) to toggle its General Purpose Input/Output (GPIO) pins at radio frequencies.

This paper builds a software-defined baseband processor in Python capable of generating arbitrary analog and digital waveforms. We first establish the hardware constraints of the Pi-FM environment. Subsequently, we detail the programmatic generation of AM and FM audio broadcasts. Finally, we execute a high-throughput 16-QAM digital transmission, evaluating the destructive impact of the physical radio channel on complex constellations.

II. HARDWARE FOUNDATION AND LIMITATIONS

The fundamental mechanism of the Raspberry Pi transmitter relies on bit-banging GPIO pins using the internal Phase-Locked Loop (PLL). To understand the physical constraints of this testbed, we refer to the foundational hardware analysis conducted by Aditya and Suraj [1].

A. System Stability and Thermal Constraints

Aditya and Suraj noted that manipulating the PLLC introduces systemic risks, as it simultaneously drives the Pi's GPU and system RAM. Furthermore, because the transmission requires precise CPU timing, the system is highly sensitive to environmental factors. During their stress testing, any thermal throttling or undervoltage immediately corrupted the carrier frequency, reducing the output to static noise [1]. Therefore, our setup enforces strict thermal management and stable power delivery to maintain transmission integrity.

B. Square Wave Harmonics

A critical physical constraint of the Pi-FM architecture is that a digital GPIO pin inherently outputs a square wave, not a continuous sine wave. As Aditya and Suraj demonstrated, broadcasting a 100 MHz square wave radiates dangerous amounts of energy at odd harmonics (e.g., 300 MHz, 500 MHz, 700 MHz) [1]. To responsibly utilise this hardware and prevent interference with external communication bands, a physical Band Pass Filter (BPF) must be inserted between the GPIO output and the antenna.

III. EVOLUTION FROM PiFM TO `RPITX`

While the baseline Pi-FM architecture established the viability of the Raspberry Pi as an RF transmitter, its capabilities were strictly confined to Frequency Modulation. To support advanced digital communication schemes, the software architecture evolved into the `rpitx` framework, introducing critical modifications to handle arbitrary Complex Baseband (I/Q) signals.

A. Hardware and Software Limitations of PiFM

PiFM operates by manipulating the fractional divider of the Raspberry Pi's Phase-Locked Loop (PLL). By continuously adjusting the division ratio via DMA, the hardware generates a carrier wave that shifts in frequency proportionally to an audio input. However, this mechanism fundamentally lacks the capacity to alter the signal's amplitude or directly control phase shifts, restricting its utility to basic analog audio broadcasts.

B. The `rpitx` Architecture: Arbitrary Modulation

Complex digital modulation, such as 16-QAM or Single-Sideband (SSB), requires the simultaneous control of both phase and amplitude (In-Phase and Quadrature components). To achieve this without a dedicated hardware DAC, `rpitx` implements two major structural changes:

- **Phase via Frequency Integration:** Instead of relying on a dedicated phase shifter, `rpitx` mathematically calculates the instantaneous phase derivative. Because phase is the mathematical integral of frequency over time ($\phi = \int f \cdot dt$), `rpitx` precisely modulates the PLL's frequency to hit the exact phase targets required by digital constellations.
- **Amplitude via Pulse Width Modulation (PWM):** To introduce amplitude variations, `rpitx` manipulates the duty cycle of the GPIO output using PWM. By rapidly switching the pulse width of the fundamental square

wave, it alters the effective drive strength of the signal, synthetically creating amplitude modulation.

By heavily utilising the DMA controller to synchronise both the PLL registers (for phase/frequency) and the PWM peripheral (for amplitude) at multi-megahertz clock speeds, `rpitx` essentially transforms the Raspberry Pi into a universal, programmable RF transmitter.

IV. ANALOG BASEBAND GENERATION

We generate the mathematical Complex Baseband (I/Q) arrays before handing them to the hardware.

A. AM and FM Modulation in Python

To broadcast analog audio, we map the varying amplitudes of a standard `.wav` file into In-Phase (I) and Quadrature (Q) arrays. For Amplitude Modulation (AM), the audio envelope simply scales the I-channel. For Frequency Modulation (FM), the integral of the audio signal dictates the instantaneous phase shift.

```
import numpy as np
from scipy.io import wavfile

def generate_am_iq(audio_file,
    out_file="am.iq"):
    fs, audio = wavfile.read(audio_file)
    audio = audio / np.max(np.abs(audio)) #
    Normalise

    # AM: Envelope modulation
    iq_am = (1 + 0.8 * audio) + 0j

    iq_am.astype(np.complex64).tofile(out_file)

def generate_fm_iq(audio_file,
    out_file="fm.iq", max_dev=75000):
    fs, audio = wavfile.read(audio_file)
    audio = audio / np.max(np.abs(audio))

    # FM: Phase is the integral of the
    message
    dt = 1.0 / fs
    phase = 2 * np.pi * max_dev *
    np.cumsum(audio) * dt
    iq_fm = np.exp(1j * phase)

    iq_fm.astype(np.complex64).tofile(out_file)
```

Listing 1. Generation of AM and FM I/Q Baseband Signals

The generated `.iq` files are then transmitted using the hardware driver, allowing any RTL-SDR receiver to demodulate the audio using standard AM/WFM protocols.

V. DIGITAL MODULATION AND MULTIPATH ANALYSIS

To advance beyond analog audio, we constructed a digital modulation pipeline. Binary data is mapped to discrete states of phase and amplitude.

A. Frequency-Shift Keying (FSK)

Frequency-Shift Keying (FSK) represents binary data through discrete changes in the frequency of the carrier wave. Similar to analog FM, we calculate the continuous phase by integrating the discrete frequency shifts.

```
import numpy as np

def generate_fsk_iq(num_symbols=1000,
    out_file="fsk.iq", deviation=5000,
    fs=48000):
    # Map binary data to frequency shifts
    (-deviation, +deviation)
    data = np.random.randint(0, 2,
    num_symbols)

    # Upsample to create continuous symbols
    in time
    sps = 10
    upsampled_data = np.repeat(data, sps)
    freq_signal = np.where(upsampled_data
    == 0, -deviation, deviation)

    # Integrate frequency to get
    instantaneous phase
    dt = 1.0 / fs
    phase = 2 * np.pi *
    np.cumsum(freq_signal) * dt

    # Generate complex baseband signal
    iq_fsk = np.exp(1j * phase)

    iq_fsk.astype(np.complex64).tofile(out_file)
```

Listing 2. 2-FSK Generation via Frequency Integration

B. Binary Phase-Shift Keying (BPSK)

Binary Phase-Shift Keying (BPSK) was implemented as the fundamental baseline for digital I/Q constellations. It maps binary zeros and ones directly to 180° phase shifts (+1 and -1 on the real axis). To confine the infinite bandwidth caused by instantaneous phase switching, a Root Raised Cosine (RRC) pulse-shaping filter was applied.

```
import numpy as np

def generate_bpsk_iq(num_symbols=1000,
    out_file="bpsk.iq"):
    # BPSK Constellation Mapping (0 ->
    -1+0j, 1 -> +1+0j)
    data = np.random.randint(0, 2,
    num_symbols)
    symbols = np.where(data == 0, -1+0j,
    1+0j)

    # Upsample (Samples per symbol = 4)
    sps = 4
    upsampled = np.zeros(len(symbols) *
    sps, dtype=complex)
    upsampled[::sps] = symbols

    # Simplified RRC Filter parameters
    beta = 0.35
    num_taps = 41
```

```

t = np.arange(-num_taps//2, num_taps//2
+ 1)

# RRC Impulse Response
h = np.sinc(t/sps) *
np.cos(np.pi*beta*t/sps) / (1 -
(2*beta*t/sps)**2)
h[num_taps//2] = 1 - beta + 4*beta/np.pi

# Apply filter and export
iq_signal = np.convolve(upsampled, h)

iq_signal.astype(np.complex64).tofile(out_file)

```

Listing 3. BPSK Mapping and RRC Pulse Shaping

The generated BPSK signals were successfully demodulated over the air with high integrity, validating both the software filtering and hardware phase control.

C. 16-QAM Theoretical Implementation

To maximise spectral efficiency, we implemented 16-Quadrature Amplitude Modulation (16-QAM), which transmits 4 bits per symbol across 16 distinct I/Q coordinates. To constrain the infinite bandwidth caused by instant digital switching, a Root Raised Cosine (RRC) pulse-shaping filter was applied.

```

import numpy as np

def generate_16qam_iq(num_symbols=1000,
out_file="16qam.iq"):
    # 16-QAM Constellation Mapping
    mapping = {
        0: -3-3j, 1: -3-1j, 2: -3+3j, 3:
        -3+1j,
        4: -1-3j, 5: -1-1j, 6: -1+3j, 7:
        -1+1j,
        8: 3-3j, 9: 3-1j, 10: 3+3j, 11:
        3+1j,
        12: 1-3j, 13: 1-1j, 14: 1+3j, 15:
        1+1j
    }

    # Generate random data and map to
    symbols
    data = np.random.randint(0, 16,
num_symbols)
    symbols = np.array([mapping[d] for d in
data])

    # Upsample (Samples per symbol = 4)
    sps = 4
    upsampled = np.zeros(len(symbols) *
sps, dtype=complex)
    upsampled[::sps] = symbols

    # Simplified RRC Filter parameters
    beta = 0.35
    num_taps = 41
    t = np.arange(-num_taps//2, num_taps//2
+ 1)

    # RRC Impulse Response

```

```

h = np.sinc(t/sps) *
np.cos(np.pi*beta*t/sps) / (1 -
(2*beta*t/sps)**2)
h[num_taps//2] = 1 - beta + 4*beta/np.pi

# Apply filter and export
iq_signal = np.convolve(upsampled, h)

iq_signal.astype(np.complex64).tofile(out_file)

```

Listing 4. 16-QAM Mapping and RRC Pulse Shaping

D. Over-The-Air Multipath Degradation

The mathematical theory and execution of the 16-QAM transmitter were entirely flawless. In a simulated, closed-loop wired environment, a GNU Radio receiver perfectly demodulated the signal, displaying 16 distinct, tightly clustered dots on the constellation sink.

However, during real-world, Over-The-Air (OTA) testing within an indoor environment, the reception drastically failed. The GNU Radio constellation exploded into a blurry, scattered cloud, making it impossible to decode the 16 individual states.

This was due to Multipath Fading. As the 16-QAM signal propagated through the room, the radio waves reflected off the surroundings. Reflected waves arriving microseconds later altered the phase of the primary signal, while destructive interference altered the amplitude. Because QAM inherently relies on the strict, unaltered preservation of both phase and amplitude to differentiate tightly packed symbols, these room reflections thoroughly scrambled the transmission. Advanced error correction may be needed to perform OTA reliably.

VI. CONCLUSION

This work successfully bridged the gap between purely theoretical digital signal processing and physical hardware deployment. Using the Pi-FM hardware architecture evaluated by Aditya and Suraj, we successfully coded and transmitted complex analog and digital Baseband arrays from scratch. Most notably, the 16-QAM deployment served as a practical demonstration of multipath fading.

REFERENCES

- [1] Appana, A. and Suraj, N., *The Raspberry Pi FM Transmitter (Pi-FM) Deep Dive: PLLC, DMA, and Signal Generation*. Hardware Analysis Report, Feb 16, 2026.